



Gradescope vs Copyleaks vs Turnitin AI grading: accuracy, false-positive rates, and plagiarism integration compared side-by-side

Computer

Canvas vs Blackboard vs Moodle vs D2L API rate limits, OAuth scopes, and webhook reliability compared for enterprise integrations

Computer

Compare LTI 1.3 implementation differences across the four platforms

Evaluate API rate limits and webhook reliability for Canvas and D2L

Build a QTI 3 migration checklist for legacy item banks

The key finding is that the four LMS platforms can all support an LTI 1.3 integration, but their operational behavior differs materially in registration, enabled Advantage services, administrative permissions, API throttling, and event delivery. Semester should therefore use **LTI 1.3/LTI Advantage as the portable baseline**, then add platform APIs only where a customer-approved use case requires them.

For QTI migration, treat the work as a controlled content-conversion program—not a file import. Preserve originals, build a semantic crosswalk, validate scoring and accessibility in the destination platform, and retain evidence for every exception. [¹⁶³³][¹⁶⁴¹]

LTI 1.3 comparison

Shared baseline

All four platforms can participate in LTI 1.3 workflows, but a customer’s actual tenant configuration determines which services are available. A tool should always discover and validate:

```
Issuer
Client ID
Deployment ID
OIDC login initiation endpoint
Authorization endpoint
JWKS endpoint
Access-token endpoint
Deep Linking availability
NRPS availability and scopes
AGS availability and scopes
Course/context and role claims
Platform-specific admin approval requirements
```

Do not assume that “LTI 1.3 supported” means all Advantage services are enabled. Third-party implementation guidance commonly lists launch support across Canvas, Blackboard, Moodle, and Brightspace, while the availability of roster sync, automatic grade sync, or Deep Linking can differ by platform configuration and tool. [¹⁶³⁶][¹¹]

Platform differences

Area	Canvas	Blackboard Learn	Moodle	D2L Brightspace	Semester integration practice
Tool registration	Admin-managed LTI Developer Key and deployment; JSON configuration supported	Admin registers LTI 1.3/Advantage tool, approves it, configures user fields and grade access	Platform/tool configuration is strongly version/admin dependent	Institution configures registered tool and permitted services	Store tenant-specific registration; never reuse deployments across tenants
Deployment isolation	Deployment IDs distinguish installations/deployments of a developer key	Deployment ID is captured after registration	Deployment/configuration varies by Moodle instance	Deployment/configuration varies by tenant	Bind every launch, access token, roster, and grade operation to tenant + deployment + course context
AGS/grade writes	Requires granted LTI Advantage permissions/scopes	Admin must enable grade-service access	Requires configuration/version support	Requires tenant service availability/configuration	Test line-item creation, score posting, retry, reconciliation, and student visibility before production
NRPS/roster	Scope and tool configuration controlled by admin	User fields/role settings influenced by setup	Version/configuration dependent	Tenant configuration dependent	Request only required roles/roster data; support a no-roster fallback
Deep Linking	Supported through configured tool capabilities	Enabled/configured by admin and tool placement	Version/configuration dependent	Tenant configuration dependent	Make content selection optional; provide manual launch fallback
Main operational risk	API/key scope and tenant-specific permissions	SaaS/hosted version and admin settings vary	Plugin/version/hosting differences and local governance	Entitlement/API policy and tenant settings	Maintain capability-discovery checklist for every customer integration

Canvas uses Developer Keys and LTI configuration, including JSON/config-URL approaches, while Blackboard's LTI 1.3 setup flow requires administrative registration, tool approval, and explicit grade-service access configuration. [^1631] Moodle documents LTI 1.3 as an OAuth 2.0/OpenID Connect

security model, but real-world service availability can be version- and configuration-dependent. [^1621]
[2]

API rate limits and webhook reliability

Rate-limit comparison

Platform	Published mechanism	What the integration must do	What not to assume
Canvas	Dynamic throttling with request cost and replenishing quota; throttled calls receive HTTP 429	Read rate-limit headers where available; use bounded concurrency, exponential backoff with jitter, idempotent writes, and queue-based bulk sync	A single fixed requests-per-minute quota for every route, tenant, or credential
D2L Brightspace	Token-bucket credit model; API calls consume credits and responses provide remaining credits, request cost, reset/retry timing; 429 on exhaustion	Use <code>X-Rate-Limit-Remaining</code> , <code>X-Request-Cost</code> , <code>X-Rate-Limit-Reset</code> , and <code>Retry-After</code> ; dynamically reduce concurrency and honor retry times	Every call has identical cost or a fixed capacity across all customer environments
Blackboard	Verify current REST/event limits in the particular Learn SaaS/hosted deployment and contract	Implement generic 429/backoff, pagination, retries, circuit breaking, and scheduled reconciliation	Uniform limits or webhook behavior across all hosting models
Moodle	Depends on Moodle version, enabled web services, plugins, hosting infrastructure, and customizations	Perform per-instance load testing within customer approval; use polling/reconciliation if event support is not contractual	A common webhook/event reliability model across all Moodle sites
Semester target	Per-tenant integration queue with adaptive rate control and delivery observability	Publish own rate limits, cost model, retry semantics, idempotency, delivery logs, signed webhooks, and replay controls	That downstream LMS events are complete, ordered, unique, or permanent

Canvas documents dynamic API throttling where each request has a cost against a replenishing quota and a throttled request returns HTTP 429. [^1642] Brightspace documents a token-bucket credit scheme and headers including remaining credits, per-request cost, reset time, and `Retry-After`; it also states that route costs may be adjusted dynamically. [^1638] Brightspace’s published FAQ states a 50,000-API-credit-per-minute bucket for its rate-limiting model and indicates that rate-limit events can be found in its System Log. [^1635]

Webhook reliability design

Do not design Semester’s grade or roster sync around an assumption that external LMS webhooks are exactly-once or reliably ordered. Use this delivery contract internally:

```
At-least-once delivery.
Events may be duplicated.
Events may arrive out of order.
Events may be delayed.
Some events may be absent.
Destination state may change outside the integration.
```

Semester’s integration processor should:

1. Verify webhook signature and timestamp when available.
2. Store raw event metadata and correlation ID.
3. Deduplicate with event ID or idempotency key.

4. Place event on a durable queue.
5. Process with tenant/deployment/course context.
6. Use optimistic concurrency/version checks where supported.
7. Retry transient failures with exponential backoff and jitter.
8. Respect 429 and Retry-After response behavior.
9. Put permanent failures into an operator-visible dead-letter queue.
10. Reconcile source-of-truth state on a scheduled basis.
11. Provide a customer-visible health dashboard: last success, lag, failed records, retry state, and manual-repair action.

Canvas and D2L evaluation test

Run this in a customer sandbox before production:

Create 100-1,000 representative course/enrollment/assignment records, within customer-approved limits.

Test:

- Read pagination and incremental sync.
- Burst and sustained-call behavior.
- 429 handling and recovery.
- Grade write idempotency.
- Duplicate event handling.
- Out-of-order update handling.
- Retry after a simulated destination outage.
- Reconciliation after a deliberately dropped event.
- Token/key expiry and rotation.
- Permission revocation.
- Student visibility after an AGS grade update.
- Audit-log completeness.

Success should mean:

No duplicate grade posted.
No missed final state after reconciliation.
No cross-tenant record access.
No unbounded retry storm.
Every failed grade sync has a visible exception and operator action.

Gradescope, Copyleaks, and Turnitin

Do not use a universal accuracy number

It would be misleading to publish one “accuracy” or “false-positive rate” for Gradescope, Copyleaks, or Turnitin. These systems address different problems:

- **Gradescope** focuses on grading workflow, rubric-based marking, answer grouping, and grading assistance.
- **Turnitin** is primarily a similarity/integrity product, with AI-writing detection available under particular product and licensing conditions.
- **Copyleaks** offers plagiarism and AI-content detection capabilities.

A false-positive rate depends on the task, academic discipline, language, student population, accommodation/use of assistive technology, model version, threshold, text length, reference corpus,

and the institution's definition of a true finding. The correct comparison is a controlled evaluation on permissioned, representative samples.

Dimension	Gradescope	Turnitin	Copyleaks	Semester policy
Primary use	Rubric-based grading and answer grouping	Similarity and AI-writing signals	Similarity and AI-content signals	Source-aware assessment, feedback, grades, and human review
Automated scoring role	Assists graders; appropriate for structured workflow with instructor oversight	Not a final-score engine	Not a final-score engine	AI may suggest evidence/feedback; human owns official grade
Plagiarism/similarity	Not its primary category	Core capability	Core capability	Integrate only after customer approval; treat as review signal
AI-writing detection	Not its central role	Product-dependent detection capability	Product-dependent detection capability	No automatic academic-misconduct accusation or penalty
Evidence available to instructor	Rubric and response grouping context	Similarity report/source matches; AI signal depends on product	Similarity/AI report depends on product	Rubric version, student work, source/citation context, AI-assist trace, reviewer decision
Student due process	Institution must configure review process	Institution must provide human review and response process	Institution must provide human review and response process	Student can see policy basis, respond, request review, and appeal
Data governance question	Retention, storage, integrations, training/data-use terms	Repository, retention, similarity corpus, AI-detection handling	Retention, repositories, model/data-use terms	No vendor training on student work by default; tenant approval required

Gradescope describes AI-assisted grading and answer groups for fixed-template PDF submissions; that supports a workflow-assistance model, not an unattended final-grading model. [1615] Copyleaks and Turnitin should be evaluated as signals requiring human review, not proof that a student used AI or committed misconduct.

Vendor test protocol

Require each vendor to participate in or support this test:

Dataset:

- Permissioned historical assignments or synthetic benchmark materials.
- Multiple disciplines and assignment types.
- Multiple language backgrounds and writing styles.
- Accessibility-tool outputs where relevant.
- Known original, cited/quoted, collaboratively authored, AI-assisted-permitted, and policy-violating examples.
- Predefined ground-truth review process with multiple human reviewers.

Measure:

- True-positive, false-positive, true-negative, false-negative counts.
- Precision, recall, false-positive rate, and confidence intervals.
- Results by discipline, text length, language, and approved assistive-tool use.
- Consistency after vendor/model/threshold updates.

- Explanation quality and reviewer agreement.
- Time saved versus additional review burden.
- Student appeal outcomes.

Do not deploy an integrity signal in a consequential workflow unless:

- A trained human reviews it.
- The student has a way to respond.
- The instructor sees evidence, not only a percentage.
- The institution has an appeal/process policy.
- The tool's data retention/training terms are contractually acceptable.
- The institution has tested impacts on multilingual and disabled students.

QTI 3 legacy item-bank migration checklist

QTI 3 migration must preserve both **content** and **meaning**: identifiers, media, accessibility alternatives, response processing, scoring, outcome mapping, metadata, and delivery behavior.

Phase 0: Establish scope and acceptance criteria

- [] Identify source LMS/item-bank products and versions.
- [] Identify source formats: QTI 1.2, QTI 2.x, proprietary XML/JSON, CSV, Word/PDF, or vendor export.
- [] Define destination authoring/delivery systems and supported QTI 3 profile.
- [] Inventory banks, tests, sections, items, pools, shared stimuli, rubrics, outcomes, media, formulas, stylesheets, and external links.
- [] Define acceptable loss per item type, scoring rule, asset type, and metadata field.
- [] Define "block migration" rules for high-stakes, inaccessible, unsupported, corrupted, or unvalidated items.
- [] Preserve immutable original exports and compute checksums.
- [] Establish migration RACI, security/access roles, and change control.

Phase 1: Inventory and classify

- [] Assign every source object a stable migration ID.
- [] Record source system, bank/test/item IDs, ownership, rights, lifecycle status, last review, usage history, and destination scope.
- [] Identify interaction type and complexity.
- [] Identify response declarations, outcome variables, feedback, partial credit, adaptive behavior, branching, timing, and scoring expressions.
- [] Inventory assets: images, audio, video, documents, equations, fonts, CSS, JavaScript, URLs, and embedded third-party content.
- [] Inventory accessibility metadata, alt text, captions, transcripts, keyboard path, language, and accommodation requirements.
- [] Classify each object: preserve exactly / map / transform with review / extension / omit with warning / block.

Phase 2: Semantic crosswalk

- [] Map source constructs to QTI 3 semantics—not merely matching XML element names.
- [] Define identifier conversion and collision-handling policy.
- [] Map interactions, response declarations, outcome declarations, response processing, feedback, templates, sections, and test parts.

- [] Map scoring, partial-credit, penalty, branching, randomization, attempt rules, and time limits.
- [] Map item metadata: outcome, topic, difficulty, cognitive level, author, owner, rights, language, review date, and source provenance.
- [] Map accessibility data and identify missing alternatives.
- [] Define supported extensions and document portability limits.
- [] Version the crosswalk and converter.

Phase 3: Convert and validate

- [] Convert a representative pilot set before bulk migration.
- [] Validate XML/schema/package structure.
- [] Validate QTI 3 semantics and destination-profile compatibility.
- [] Run scoring fixtures: expected response → expected score/outcome.
- [] Verify partial credit, negative scoring, randomization, feedback, branching, and outcome reporting.
- [] Verify every asset is packaged, rights-cleared, correctly linked, and rendered without broken references.
- [] Verify equations, tables, code, media, and external URLs.
- [] Compare source and destination object counts and checksums.
- [] Produce machine-readable diagnostics and a human exception queue.

Phase 4: Accessibility and interaction validation

- [] Test keyboard-only completion.
- [] Test screen-reader labels, instructions, focus order, and response state.
- [] Test contrast, zoom, reflow, and mobile behavior where supported.
- [] Verify image descriptions, captioning, transcripts, and data-table alternatives.
- [] Verify accessible math input/rendering.
- [] Verify each drag, hotspot, graph, simulation, or media interaction has an equivalent accessible route.
- [] Test accommodation timing, attempts, and alternate-format workflows.
- [] Record known limitations and either remediate, provide an approved alternative, or block the item from use.

QTI 3's migration and implementation guidance is intended to support exchange among authoring tools, item banks, test-construction tools, delivery systems, scoring engines, and learning platforms.
^[1633]_[1603]

Phase 5: Destination round-trip and acceptance

- [] Import package into the destination item bank.
- [] Preview as author, instructor, student, keyboard user, and screen-reader user.
- [] Deliver representative assessments in a non-production course.
- [] Score actual test attempts against expected fixtures.
- [] Validate gradebook/result reporting and outcome mapping.
- [] Export clean QTI 3 from the destination.
- [] Independently validate the exported package.
- [] Compare original → migrated → re-exported identifiers, content, interactions, outcomes, scores, assets, and metadata.
- [] Reconcile all exceptions and obtain subject-matter/accessibility approval.
- [] Archive original, conversion record, test evidence, exception decisions, final export, and acceptance sign-off.

The 1EdTech migration guidance emphasizes preserving originals, inventorying interaction/scoring/accessibility behavior, creating a semantic crosswalk, using diagnostics and an exception queue, validating in the actual destination workflow, and retaining a validated QTI 3 export as part of the migration record. [^1641]

Semester implementation decision

For Semester:

LTI 1.3/LTI Advantage:
The universal LMS integration baseline.

Platform APIs:
Optional, customer-approved extensions only.

External event delivery:
Helpful, never solely trusted; always reconcile.

Grade writes:
Previewed, idempotent, retry-safe, audited, and reconcilable.

AI grading/integrity:
Human review, source evidence, policy context, student response, and appeal.

QTI 3:
The native assessment portability format, with accessibility and scoring validation built into the release process.

That combination allows Semester to work across Canvas, Blackboard, Moodle, and Brightspace without inheriting the weakest operational assumptions of any one platform.

*
**

1. <https://support.tophat.com/article/Educator-introduction-to-lti13>
2. https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.lti_dev_key_config
3. <https://community.d2l.com/brightspace/kb/articles/1191-api-rate-logging-limiting-faq>
4. <https://www.lablearning.com/lti-integration>
5. <https://support.learninglink.oup.com/instructor-support/using-oll-lti-advantage/>
6. <https://docs.valence.desire2learn.com/basic/apicall.html>
7. <https://scaffold.ac/docs/admin/lti/>
8. <https://www.teachfloor.com/lti>
9. <https://www.longsight.com/guides/qti-3-migration/>
10. <https://developerdocs.instructure.com/services/canvas/basics/file.throttling>
11. <https://www.1edtech.org/standards/qti/index>
12. <https://www.1edtech.org/blog/finally-final-the-qti-30-release>
13. <https://www.imsglobal.org/spec/qti/v3p0/impl>
14. <https://guides.gradescope.com/hc/en-us/articles/24838908062093-AI-assisted-grading-and-answer-groups>
15. https://docs.moodle.org/en/Publish_as_LTI_tool
16. <https://learn.microsoft.com/en-us/linkedin/learning/sso-auth/sso-docs/sso-lti-13-blackboard>

17. https://aps.autodesk.com/en/docs/webhooks/v1/developers_guide/rate-limits/webhooks-rate-limits
18. <https://www.msglobal.org/spec/qti/v3p0/guide>
19. https://www.msglobal.org/sites/default/files/spec/qti/v3/info/imsqti_asi_v3p0p1_infomodel_v1p0.html